Web Programming

Rodrigo Martins Pagliares

Last update: 11/01/2025

Web Programming

Rodrigo Martins Pagliares

JavaScript
Dynamic client-side scripting

JavaScript first steps

Basic math in JavaScript

(numbers and operators)

Lecture: Basic math in JavaScript - numbers and operators

Module: JavaScript first steps

Objective of the lecture

Objective: To gain **familiarity** with the **basics** of **math** in **JavaScript**.

Introduction

Introduction

- In this **lecture**, we will **discuss** how we can use **operators** and **other features** to **manipulate numbers**.

Everybody  math!

Types of numbers

- **JavaScript** only **has one data type** for **numbers**, both **integers** and **decimals:**
  - ✓`Number`

Note

JavaScript has a second number type, **`BigInt`**, used for very, **very large integers**. But for the **purposes** of this **course**, we'll just worry about **`Number`** values.

BigInt

Jump to section

`BigInt` is a built-in object that provides a way to represent whole numbers larger than $2^{53} - 1$, which is the largest number JavaScript can reliably represent with the `Number` primitive and represented by the `Number.MAX_SAFE_INTEGER` constant. **`BigInt`** can be used for arbitrarily large integers.

Description

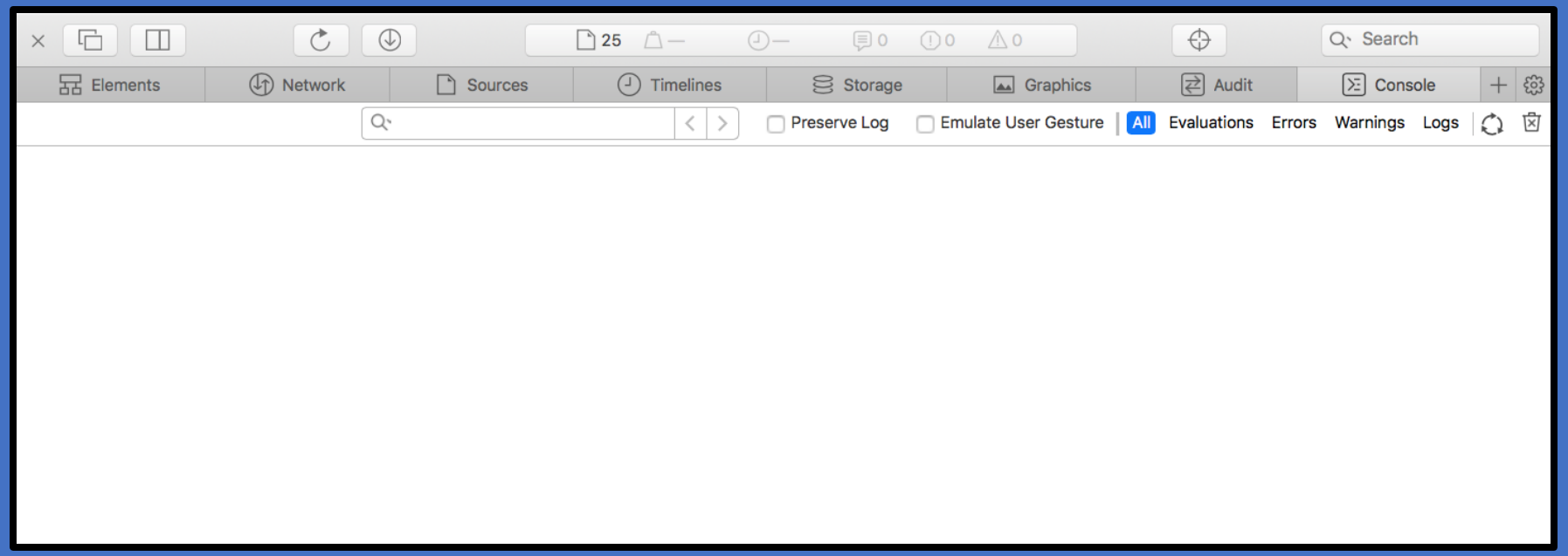
A `BigInt` is created by appending `n` to the end of an integer literal — `10n` — or by calling the function `BigInt()`.

```
1  const theBiggestInt = 9007199254740991n
2
3  const alsoHuge = BigInt(9007199254740991)
4  // ↪ 9007199254740991n
5
6  const hugeString = BigInt("9007199254740991")
7  // ↪ 9007199254740991n
```

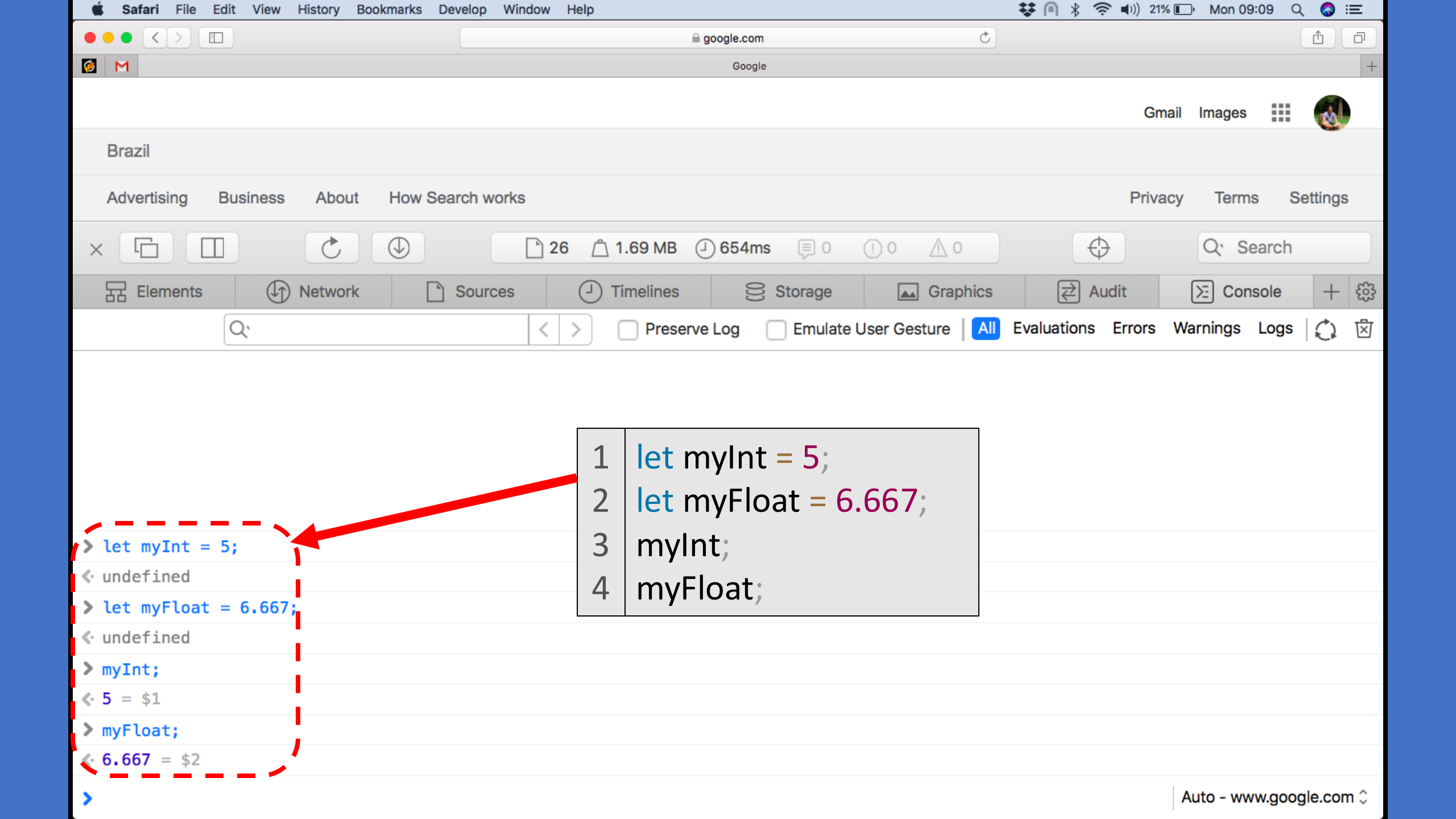
https://mzl.la/3iT9EIp

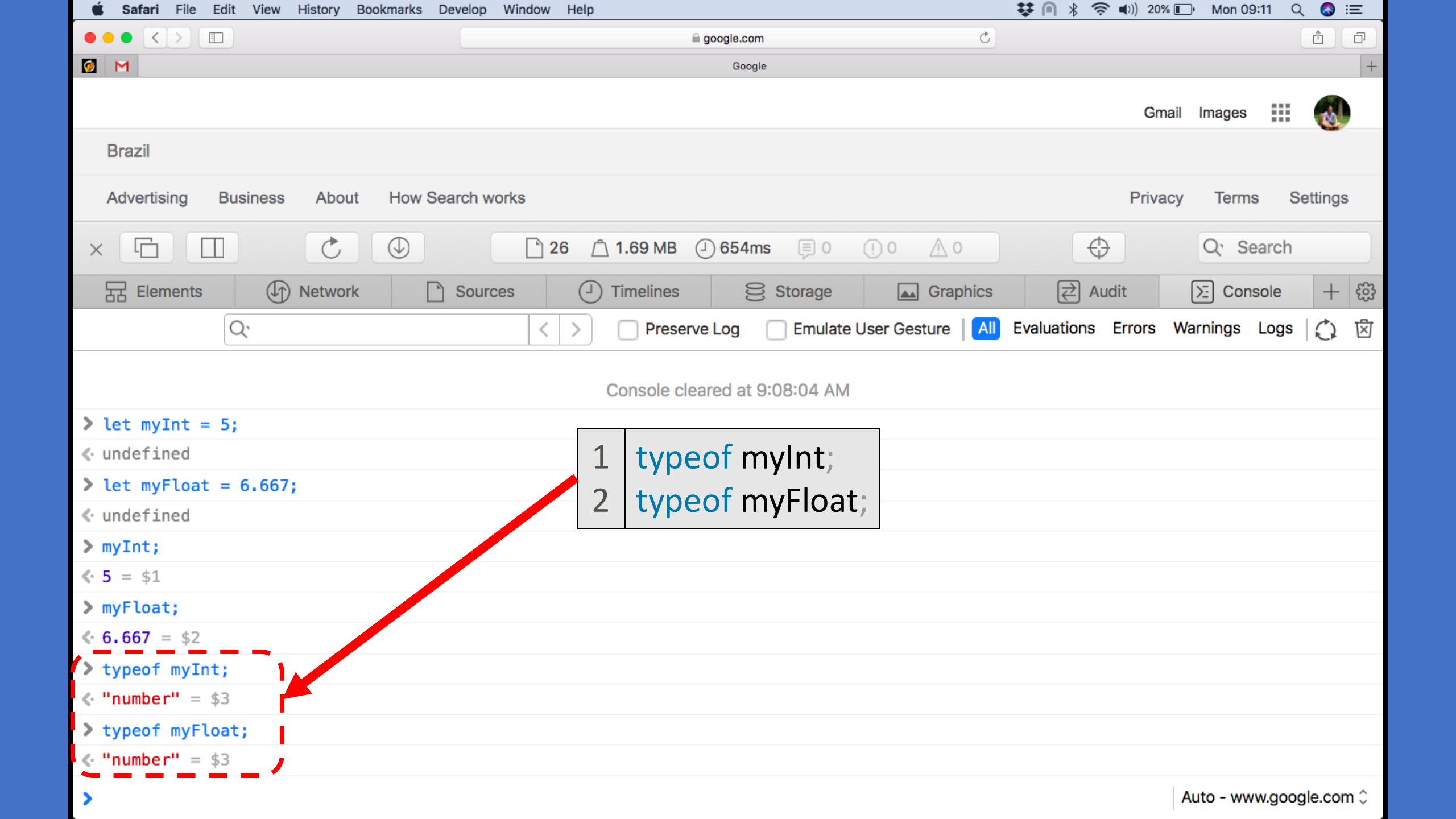
It’s all numbers to me

- **Enter** the **commands** listed in this **lecture** into your **developer’s tool JavaScript Console**.



Console in Safari developer’s tool

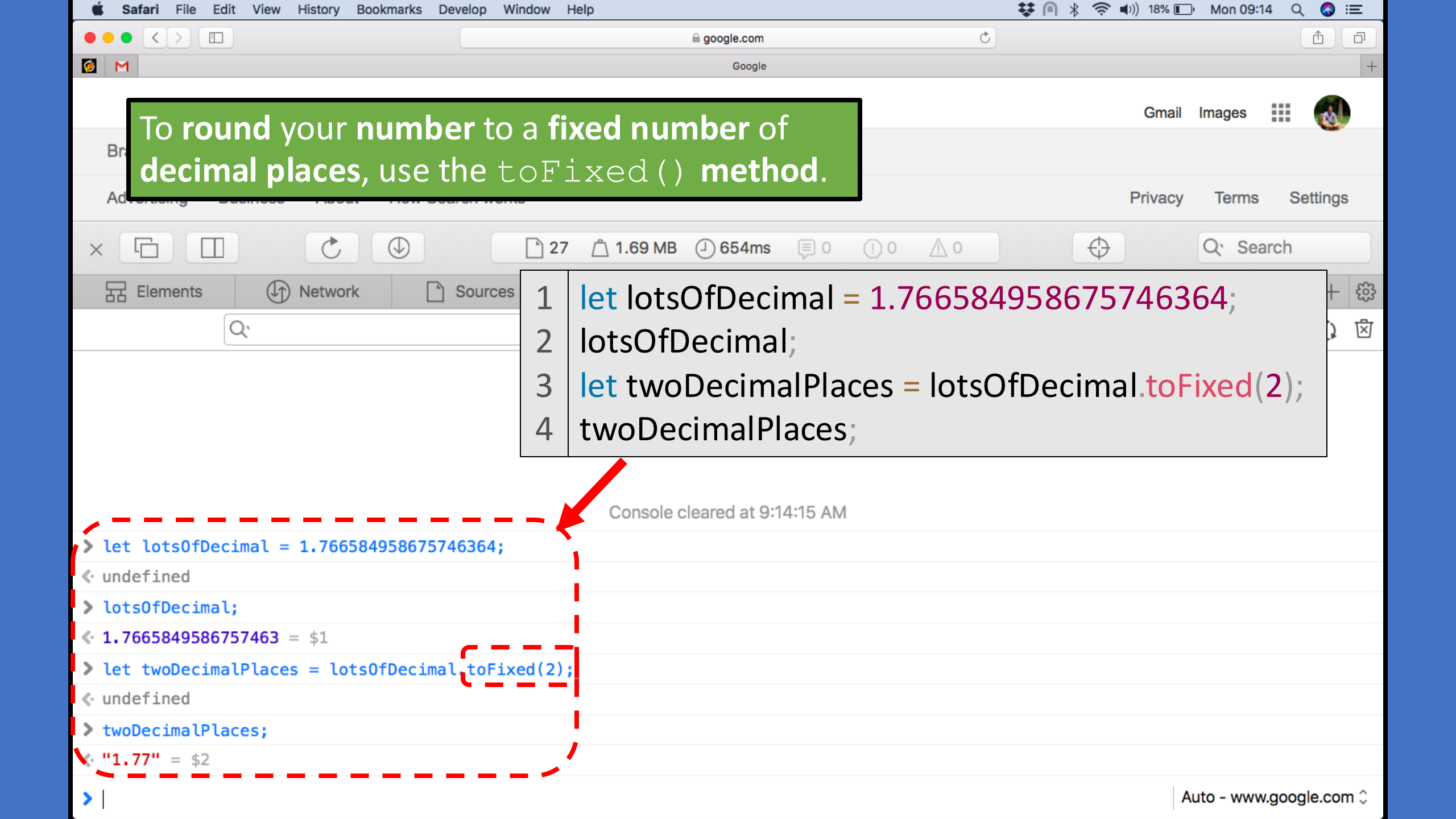

Safari File Edit View History Bookmarks Develop Window Help
21% Mon 09:09
google.com
Google
Gmail Images
Brazil
Advertising Business About How Search works
Privacy Terms Settings
26 1.69 MB 654ms 0 0 0
Search
Elements Network Sources Timelines Storage Graphics Audit Console
Preserve Log Emulate User Gesture All Evaluations Errors Warnings Logs
1 let myInt = 5;
2 let myFloat = 6.667;
3 myInt;
4 myFloat;
> let myInt = 5;
undefined
> let myFloat = 6.667;
undefined
> myInt;
5 = $1
> myFloat;
6.667 = $2
Auto - www.google.com


Safari File Edit View History Bookmarks Develop Window Help
20% Mon 09:11
google.com
Google
Gmail Images
Brazil
Advertising Business About How Search works
Privacy Terms Settings
26 1.69 MB 654ms 0 0 0
Search
Elements Network Sources Timelines Storage Graphics Audit Console
Preserve Log Emulate User Gesture All Evaluations Errors Warnings Logs
Console cleared at 9:08:04 AM
> let myInt = 5;
undefined
> let myFloat = 6.667;
undefined
> myInt;
5 = $1
> myFloat;
6.667 = $2
> typeof myInt;
"number" = $3
> typeof myFloat;
"number" = $3
1 typeof myInt;
2 typeof myFloat;
Auto - www.google.com

Useful `Number` methods

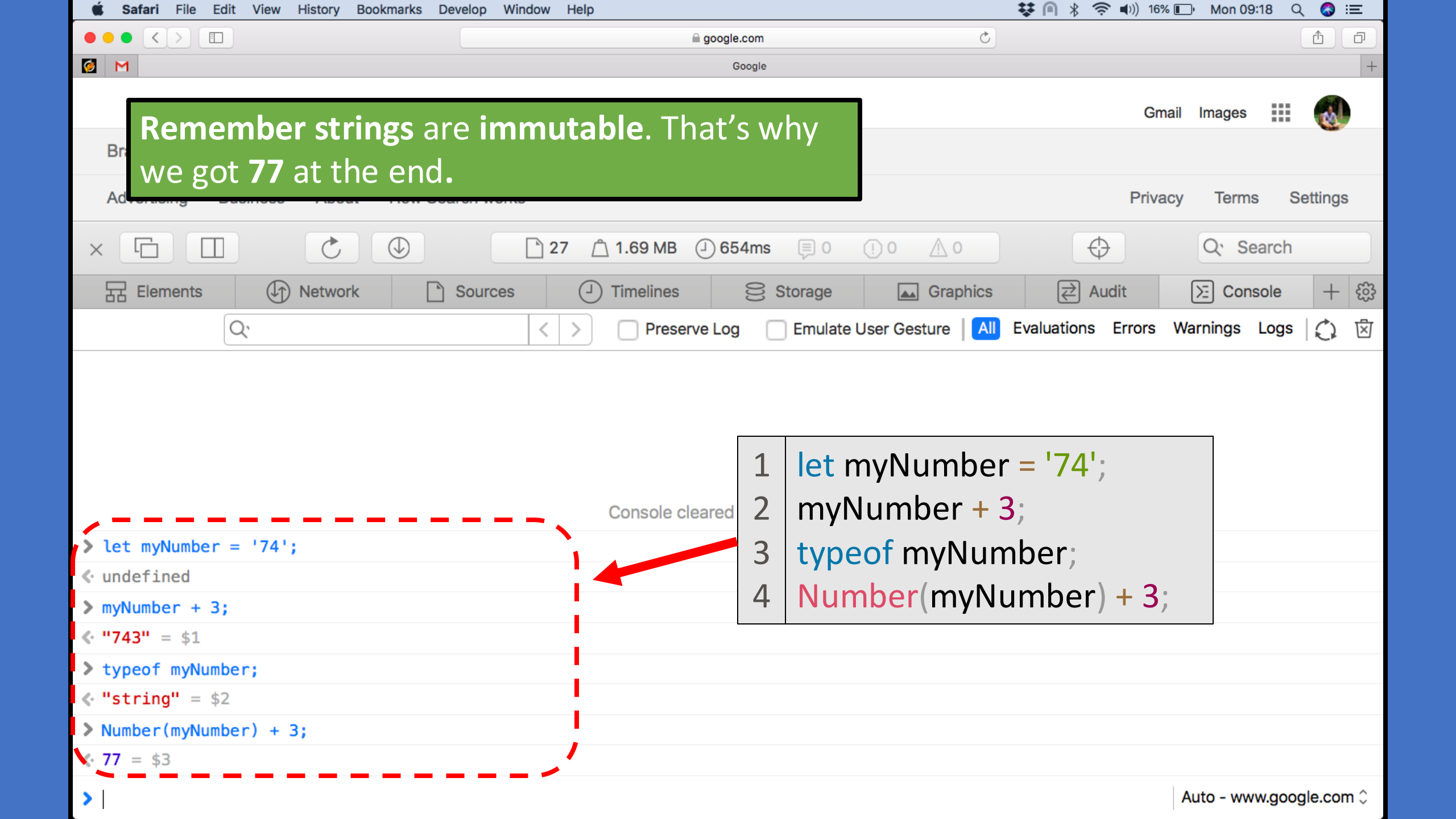
- The `Number` **object is** an **instance** of which **represents all** standard **numbers** you'll use in your **JavaScript.**

- The **number object has** useful **methods available** on it for you to **manipulate numbers**.


To round your number to a fixed number of decimal places, use the toFixed() method.
1 let lotsOfDecimal = 1.7665849586757463;
2 lotsOfDecimal;
3 let twoDecimalPlaces = lotsOfDecimal.toFixed(2);
4 twoDecimalPlaces;
Console cleared at 9:14:15 AM
> let lotsOfDecimal = 1.7665849586757463;
< undefined
> lotsOfDecimal;
< 1.7665849586757463 = $1
> let twoDecimalPlaces = lotsOfDecimal.toFixed(2);
< undefined
> twoDecimalPlaces;
< "1.77" = $2

Converting to number data types

- **Sometimes** you might **end up** with a **number** that is **stored** as a **string type**.
- This most **commonly happens** when **data** is **entered** into a **form input**, and the **input type is text**.
- **There is** a **way** to **solve** this **problem**
  - ✓**Passing** the **string value** into the `Number()` **constructor** to **return** a **number version** of the **same value**.


Safari File Edit View History Bookmarks Develop Window Help
16% Mon 09:18
google.com
Google
Gmail Images
Privacy Terms Settings
Remember strings are immutable. That's why we got 77 at the end.
27 1.69 MB 654ms 0 0 0
Search
Elements Network Sources Timelines Storage Graphics Audit Console
Preserve Log Emulate User Gesture All Evaluations Errors Warnings Logs
Console cleared
> let myNumber = '74';
undefined
> myNumber + 3;
"743" = $1
> typeof myNumber;
"string" = $2
> Number(myNumber) + 3;
77 = $3
Auto - www.google.com
1 let myNumber = '74';
2 myNumber + 3;
3 typeof myNumber;
4 Number(myNumber) + 3;

developer.mozilla.org

BigInt

Jump to section

Search

Elements Network Console

Errors Warnings Logs

Remember strings are **immutable**. That's why we got **77** and not **746** in the **previous slide**. If our intention it to get **746**, we could use this **code instead**.

```
> let myNumber = '74';
< undefined
> let newNumber = myNumber + 3;
< undefined
> Number(newNumber) + 3;
< 746 = $1
>
```

```
1  let myNumber = '74';
2  let newNumber = myNumber + 3;
3  Number(newNumber) + 3;
```

Auto - BigInt

Arithmetic operators

Arithmetic operators

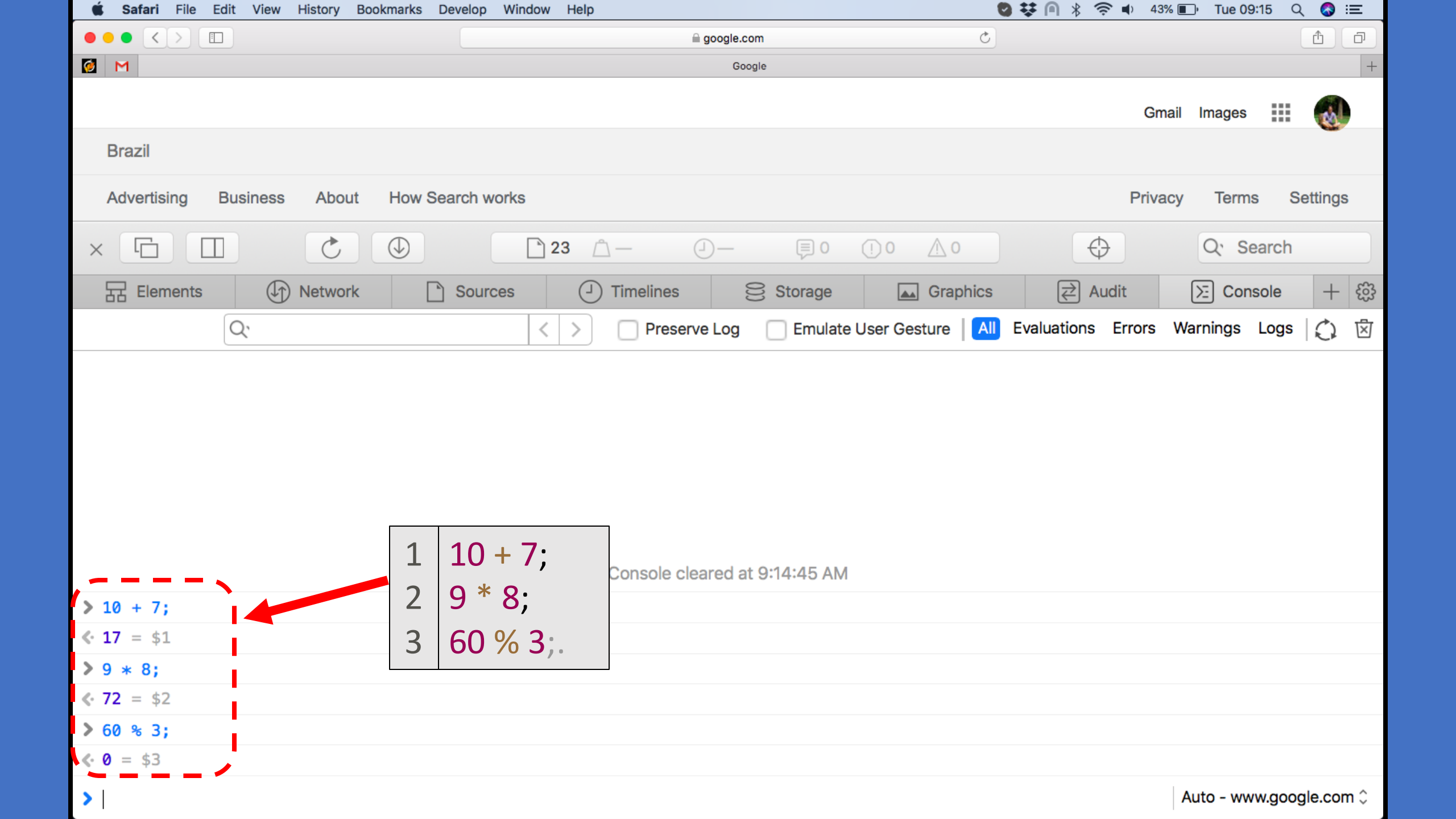
Operator	Name	Purpose	Example
`+`	Addition	Adds two numbers together.	`6 + 9`
`-`	Subtraction	Subtracts the right number from the left.	`20 - 15`
`*`	Multiplication	Multiplies two numbers together.	`3 * 7`
`/`	Division	Divides the left number by the right.	`10 / 5`
`%`	Remainder (sometimes called modulo)	Returns the remainder left over after you've divided the left number into a number of integer portions equal to the right number.	`8 % 3` (returns 2, as three goes into 8 twice, leaving 2 left over).
`**`	Exponent	Raises a `base` number to the `exponent` power, that is, the `base` number multiplied by itself, `exponent` times. It was first Introduced in EcmaScript 2016.	`5 ** 2` (returns `25`, which is the same as `5 * 5`).

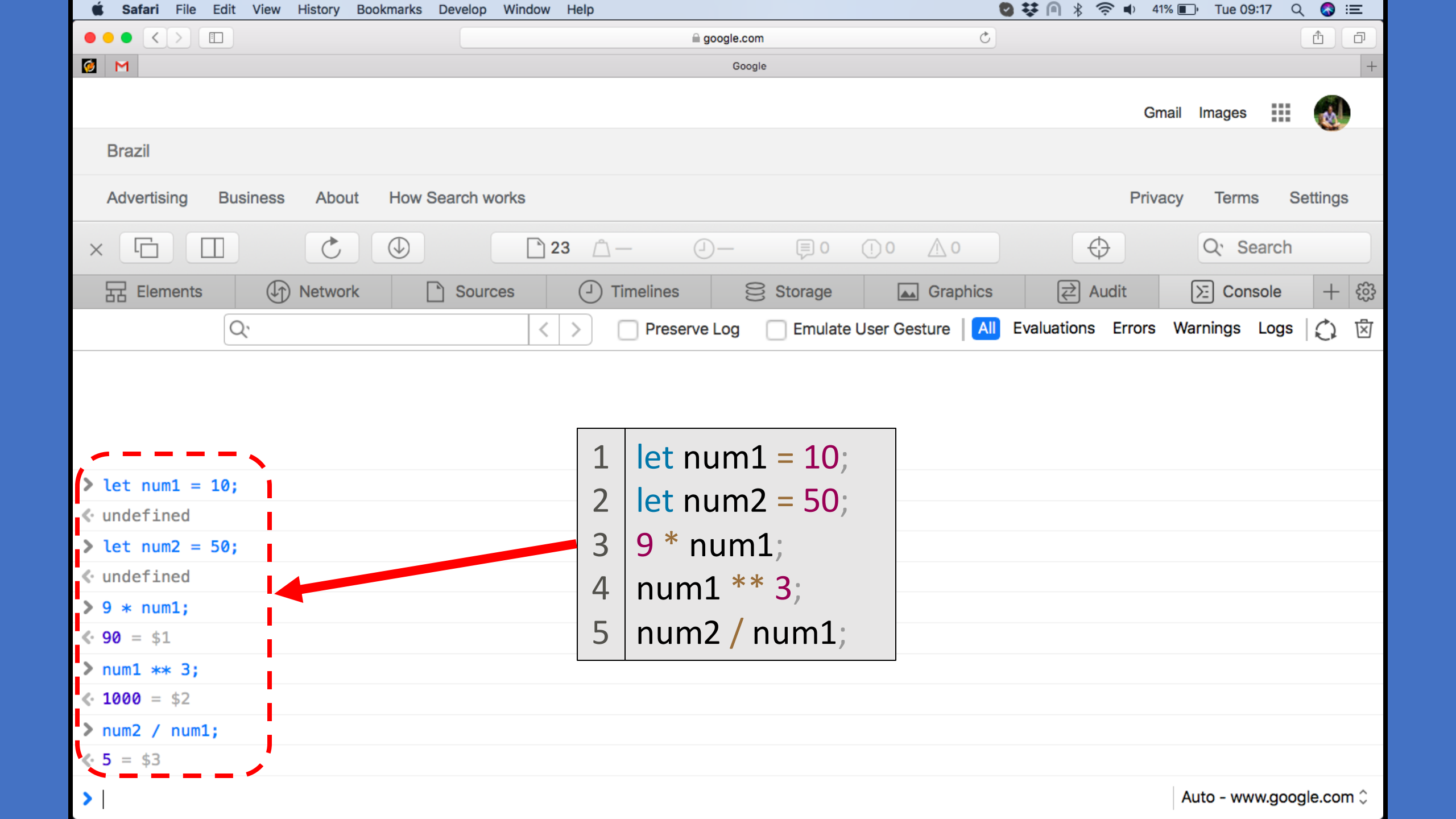
Notes

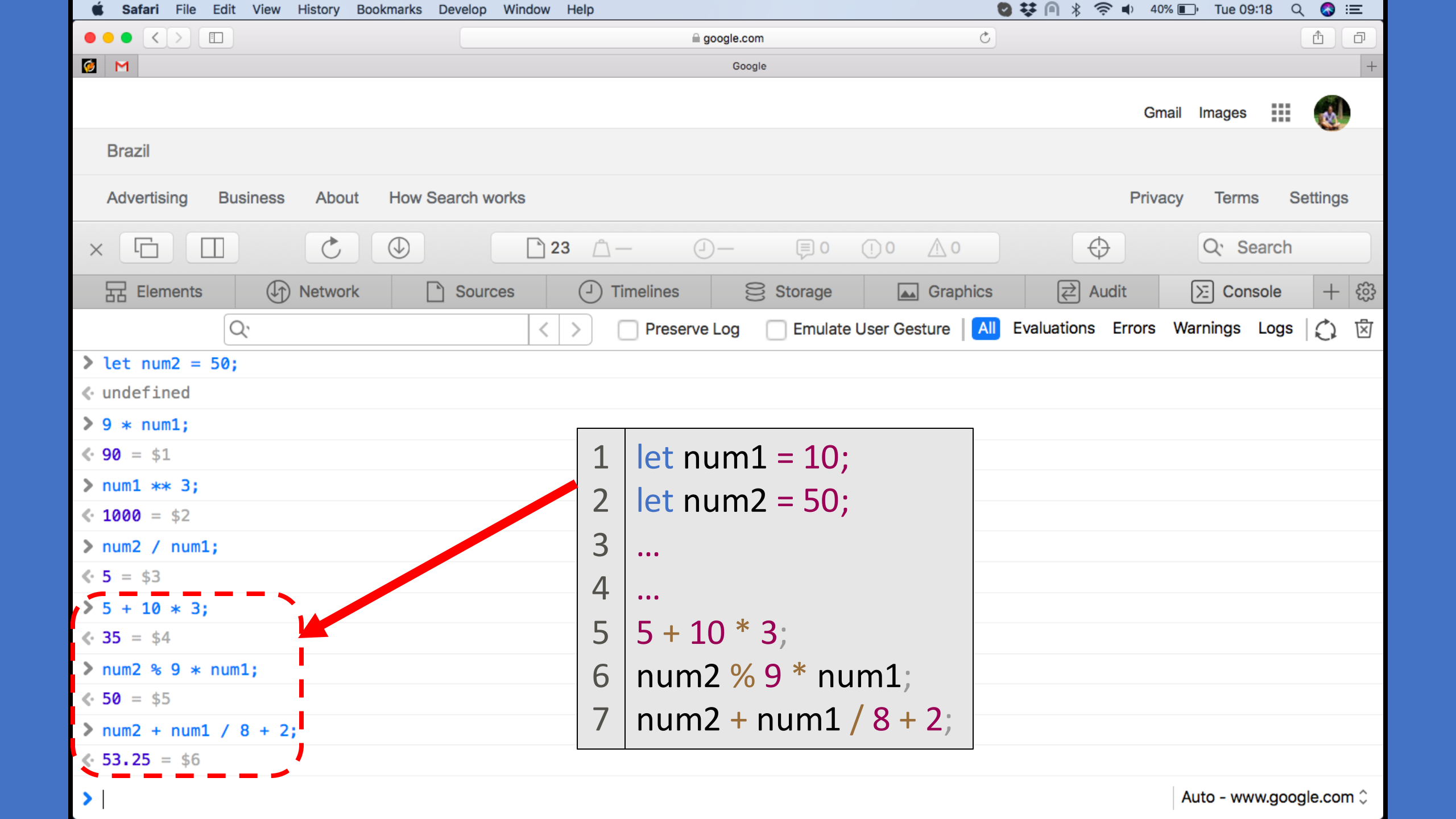
You'll **sometimes** see **numbers** involved in **arithmetic** referred to as **operands**.

Notes

You **may** sometimes **see exponents** expressed using the older `Math.pow()` **method**, which **works** in a very **similar** way. For example, in `Math.pow(7,3)`, `7` is the **base** and `3` is the **exponent**, so the result of the **expression** is 343. `Math.pow(7,3)` is **equivalent** to `7**3`.


Safari File Edit View History Bookmarks Develop Window Help
43% Tue 09:15
google.com
Google
Gmail Images
Brazil
Advertising Business About How Search works
Privacy Terms Settings
Search
Elements Network Sources Timelines Storage Graphics Audit Console
Preserve Log Emulate User Gesture All Evaluations Errors Warnings Logs
1 10 + 7;
2 9 * 8;
3 60 % 3;.
Console cleared at 9:14:45 AM
> 10 + 7;
< 17 = $1
> 9 * 8;
< 72 = $2
> 60 % 3;
< 0 = $3
Auto - www.google.com


Safari File Edit View History Bookmarks Develop Window Help
41% Tue 09:17
google.com
Google
Gmail Images
Brazil
Advertising Business About How Search works
Privacy Terms Settings
Search
Elements Network Sources Timelines Storage Graphics Audit Console
Preserve Log Emulate User Gesture All Evaluations Errors Warnings Logs
> let num1 = 10;
< undefined
> let num2 = 50;
< undefined
> 9 * num1;
< 90 = $1
> num1 ** 3;
< 1000 = $2
> num2 / num1;
< 5 = $3
1 let num1 = 10;
2 let num2 = 50;
3 9 * num1;
4 num1 ** 3;
5 num2 / num1;
Auto - www.google.com


Safari File Edit View History Bookmarks Develop Window Help
40% Tue 09:18
google.com
Google
Gmail Images
Brazil
Advertising Business About How Search works
Privacy Terms Settings
Search
Elements Network Sources Timelines Storage Graphics Audit Console
Preserve Log Emulate User Gesture All Evaluations Errors Warnings Logs
> let num2 = 50;
< undefined
> 9 * num1;
< 90 = $1
> num1 ** 3;
< 1000 = $2
> num2 / num1;
< 5 = $3
> 5 + 10 * 3;
< 35 = $4
> num2 % 9 * num1;
< 50 = $5
> num2 + num1 / 8 + 2;
< 53.25 = $6
1 let num1 = 10;
2 let num2 = 50;
3 ...
4 ...
5 5 + 10 * 3;
6 num2 % 9 * num1;
7 num2 + num1 / 8 + 2;
Auto - www.google.com

Operator precedence

- As a **human being**, you may **read** this as *"50 plus 10 equals 60"*, then *"8 plus 2 equals 10"*, and finally *"60 divided by 10 equals 6"*.

```
1  let num1 = 10;
2  let num2 = 50;
3  num2 + num1 / 8 + 2
```

- But the **browser** does *"10 divided by 8 equals 1.25"*, then *"50 plus 1.25 plus 2 equals 53.25"*.

Operator precedence

- If you want to **override operator precedence**, you can put **parentheses around** the **parts** that you **want** to be **explicitly** dealt with **first**.

```
1  let num1 = 10;
2  let num2 = 50;
3  (num2 + num1) / (8 + 2);
```

Note

A **full list** of all **JavaScript operators** and their **precedence** can be **found** in the article **Expressions** and **operators**.

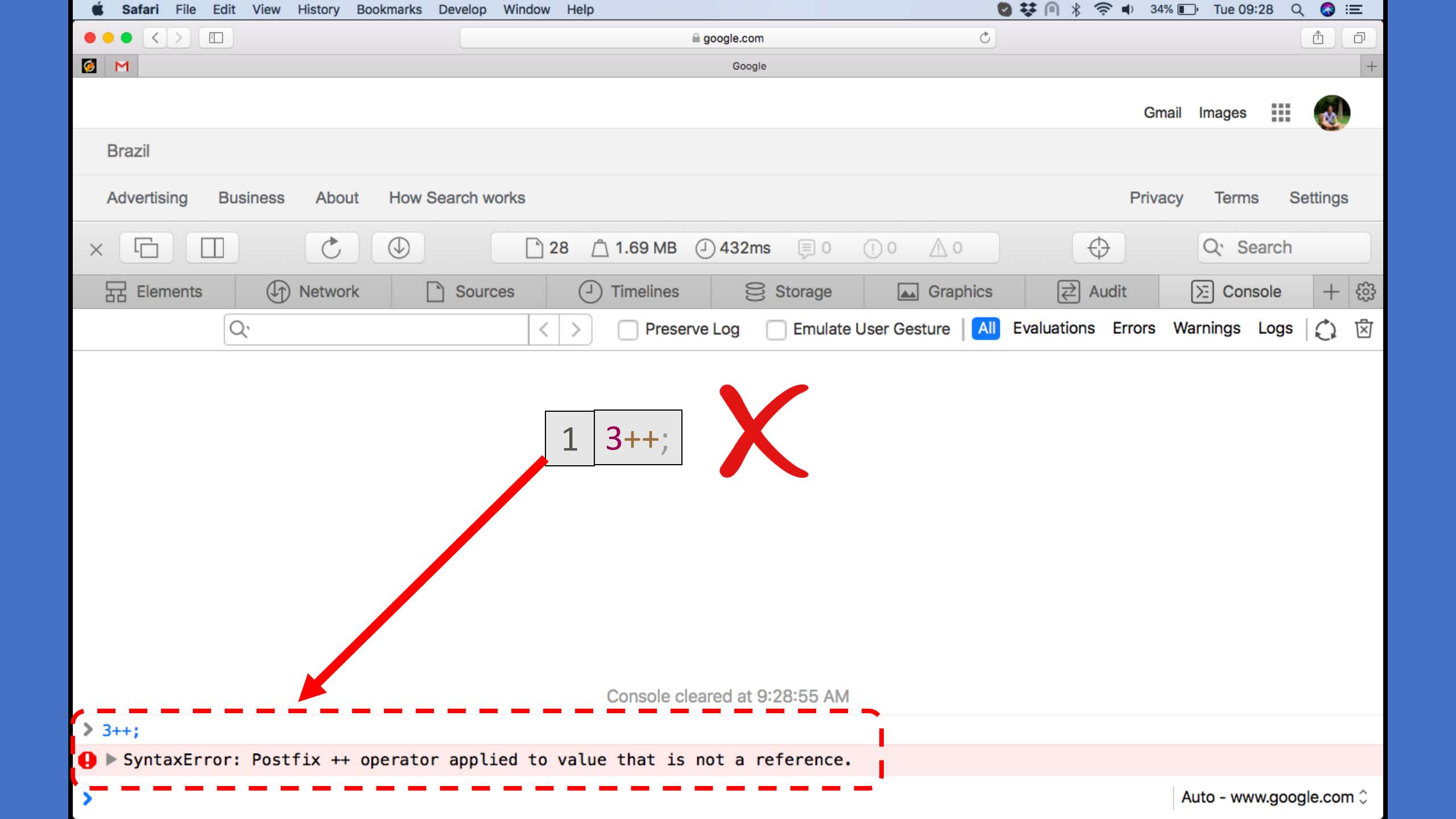
Operator precedence

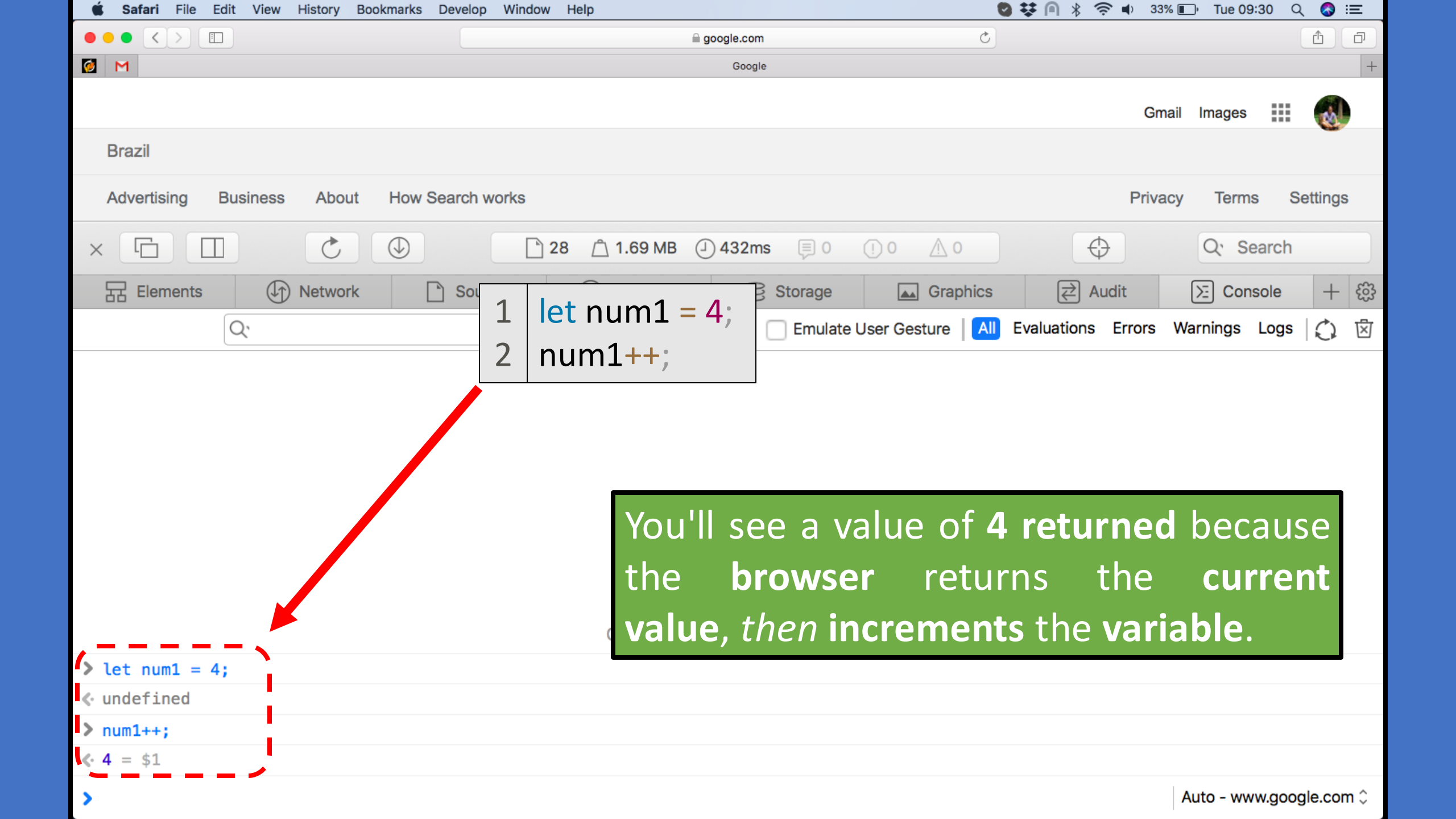
Operator type	Individual operators
member	`. []`
call / create instance	`() new`
negation/increment	`! ~ - + ++ -- typeof void delete`
multiply/divide	`* / %`
addition/subtraction	`+ -`
bitwise shift	`<< >> >>>`
relational	`< <= > >= in instanceof`
equality	`== != === !==`
bitwise-and	`&`
bitwise-xor	`^`
bitwise-or	`|`
logical-and	`&&`
logical-or	`||`
conditional	`?:`
assignment	`= += -= *= /= %= <<= >>= >>>= &= ^= |= &&= ||= ??=`
comma	`,`

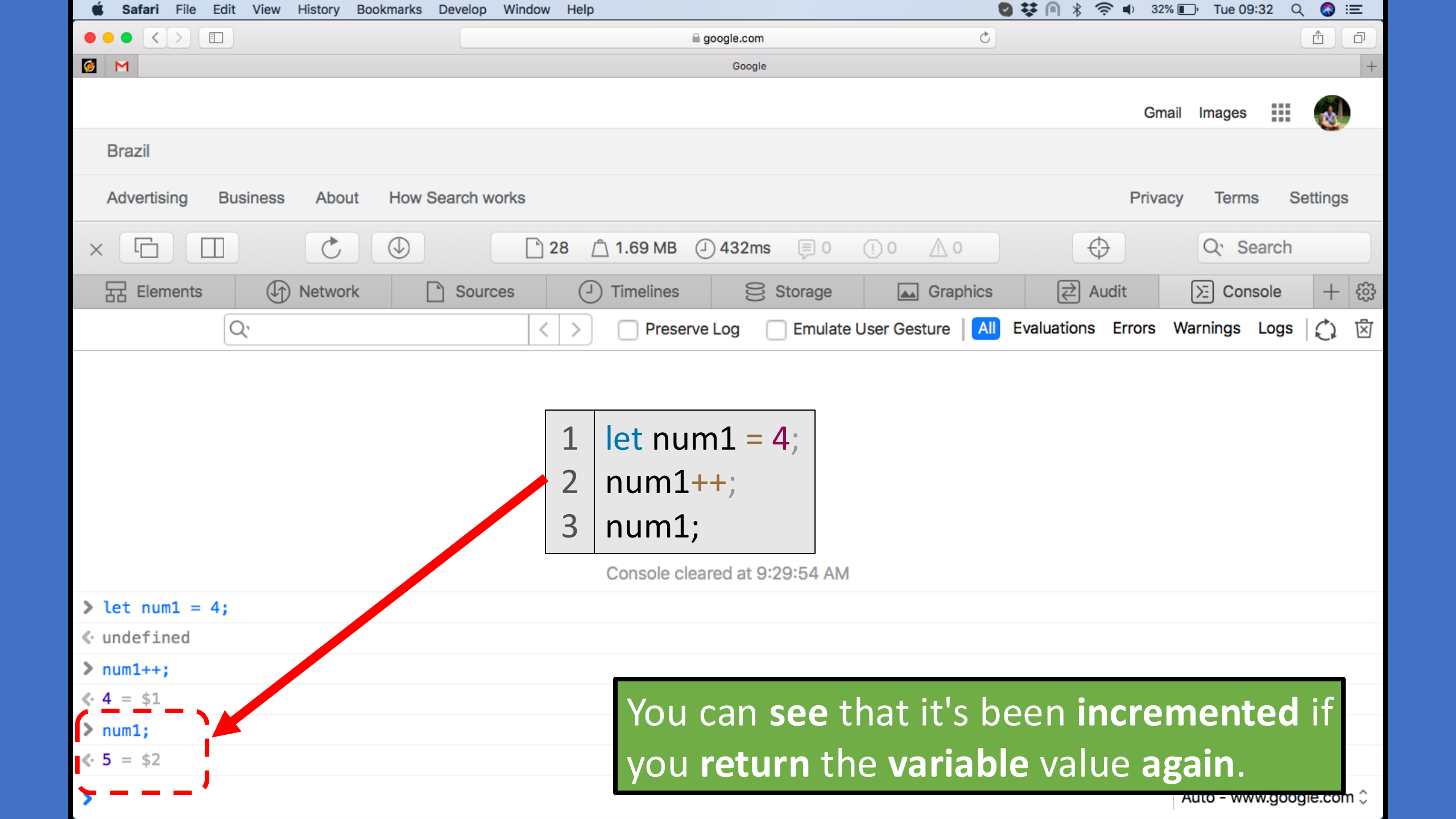
A more detailed version of this table, complete with links to additional
may be found in JavaScript Reference.

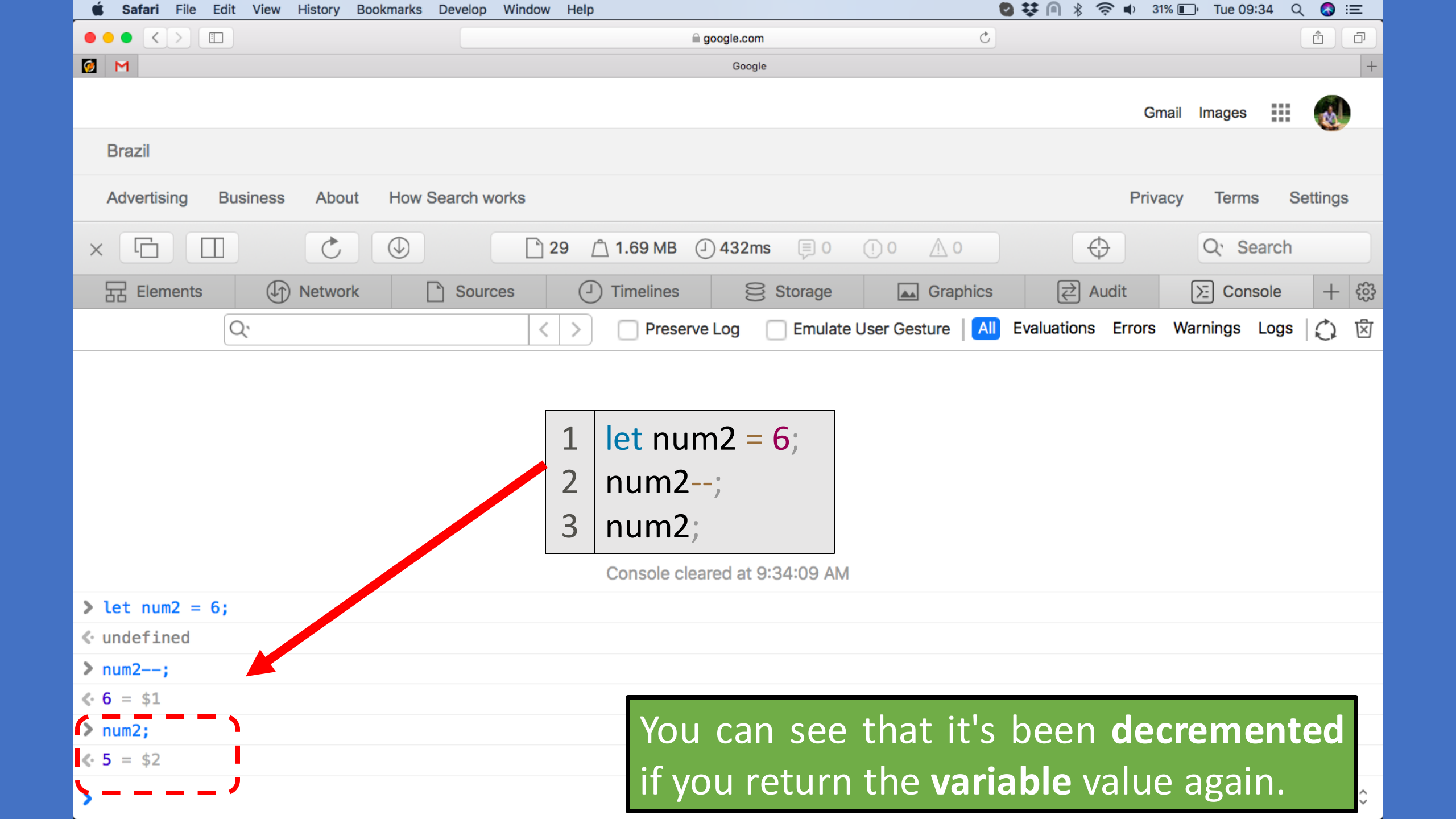
https://mzl.la/2ADgJVL

Increment & decrement operators


Safari File Edit View History Bookmarks Develop Window Help
34% Tue 09:28
google.com
Google
Gmail Images
Brazil
Advertising Business About How Search works
Privacy Terms Settings
28 1.69 MB 432ms 0 0 0
Search
Elements Network Sources Timelines Storage Graphics Audit Console
Preserve Log Emulate User Gesture All Evaluations Errors Warnings Logs
1 3++;
Console cleared at 9:28:55 AM
3++;
SyntaxError: Postfix ++ operator applied to value that is not a reference.
Auto - www.google.com


Safari File Edit View History Bookmarks Develop Window Help
33% Tue 09:30
google.com
Google
Gmail Images
Brazil
Advertising Business About How Search works
Privacy Terms Settings
28 1.69 MB 432ms 0 0 0
Search
Elements Network Storage Graphics Audit Console
Emulate User Gesture All Evaluations Errors Warnings Logs
1 let num1 = 4;
2 num1++;
You'll see a value of 4 returned because the browser returns the current value, then increments the variable.
let num1 = 4;
undefined
num1++;
4 = $1
Auto - www.google.com


Safari File Edit View History Bookmarks Develop Window Help
32% Tue 09:32
google.com
Google
Gmail Images
Brazil
Advertising Business About How Search works
Privacy Terms Settings
28 1.69 MB 432ms 0 0 0
Search
Elements Network Sources Timelines Storage Graphics Audit Console
Preserve Log Emulate User Gesture All Evaluations Errors Warnings Logs
1 let num1 = 4;
2 num1++;
3 num1;
Console cleared at 9:29:54 AM
> let num1 = 4;
undefined
> num1++;
4 = $1
> num1;
5 = $2
You can see that it's been incremented if you return the variable value again.


Safari File Edit View History Bookmarks Develop Window Help
google.com
Google
Gmail Images
Brazil
Advertising Business About How Search works
Privacy Terms Settings
29 1.69 MB 432ms 0 0 0
Search
Elements Network Sources Timelines Storage Graphics Audit Console
Preserve Log Emulate User Gesture All Evaluations Errors Warnings Logs
1 let num2 = 6;
2 num2--;
3 num2;
Console cleared at 9:34:09 AM
> let num2 = 6;
undefined
> num2--;
6 = $1
> num2;
5 = $2
You can see that it's been decremented if you return the variable value again.

Note

You can **make** the **browser** do it the other way round, **increment/decrement** the **variable** ***then*** **return** the **value,** by **putting** the **operator** at the **start** of the **variable instead** of the **end**. Try the **previous examples** again, but this time use `++num1` and `--num2`.

Assignment operators

Assignment operators

- **Operators** that **assign** a **value** to a **variable**.

```
1  let x = 3; // x contains the value 3
2  let y = 4; // y contains the value 4
3  x = y; // x now contains the same value y contains, 4
```

Assignment operators

Operator	Name	Purpose	Example	Shortcut for
`+=`	Addition assignment	Adds the value on the right to the variable value on the left, then returns the new variable value	`x += 4;`	`x = x + 4;`
`-=`	Subtraction assignment	Subtracts the value on the right from the variable value on the left, and returns the new variable value	`x -= 3;`	`x = x - 3;`
`*=`	Multiplication assignment	Multiplies the variable value on the left by the value on the right, and returns the new variable value	`x *= 3;`	`x = x * 3;`
`/=`	Division assignment	Divides the variable value on the left by the value on the right, and returns the new variable value	`x /= 5;`	`x = x / 5;`

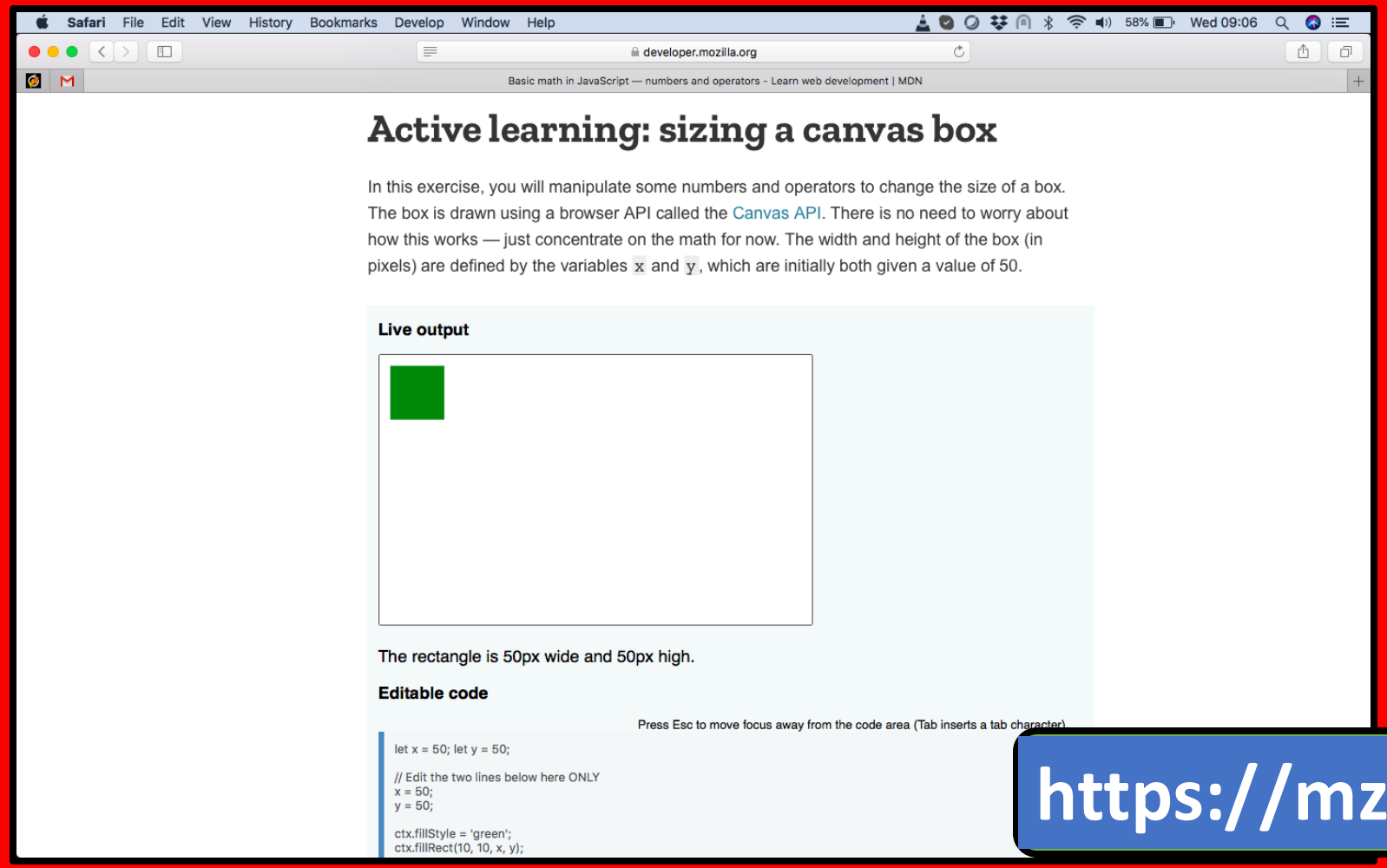
Assignment operators

```
1  let x = 3; // x contains the value 3
2  let y = 4; // y contains the value 4
3  x *= y; // x now contains the value 12
```

Operator	Name	Purpose	Example	Shortcut for
`+=`	Addition assignment	Adds the value on the right to the variable value on the left, then returns the new variable value	`x += 4;`	`x = x + 4;`
`-=`	Subtraction assignment	Subtracts the value on the right from the variable value on the left, and returns the new variable value	`x -= 3;`	`x = x - 3;`
`*=`	Multiplication assignment	Multiplies the variable value on the left by the value on the right, and returns the new variable value	`x *= 3;`	`x = x * 3;`
`/=`	Division assignment	Divides the variable value on the left by the value on the right, and returns the new variable value	`x /= 5;`	`x = x / 5;`

Active learning:
Sizing a canvas box

Go to the **link** indicated **below** for the **Active Learning statement**



https://mzl.la/3bPx1Ie

Comparison operators

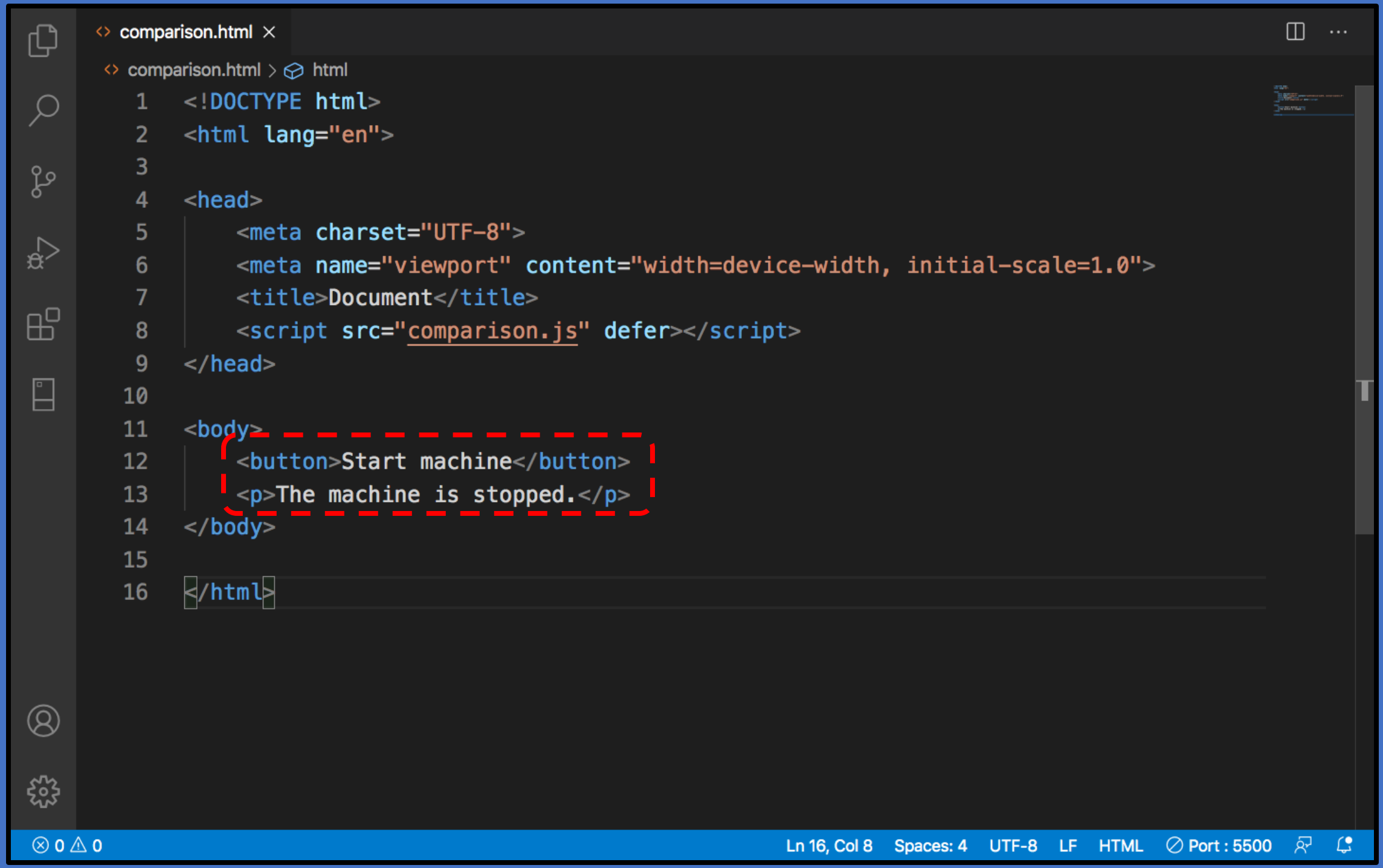
Comparison operators

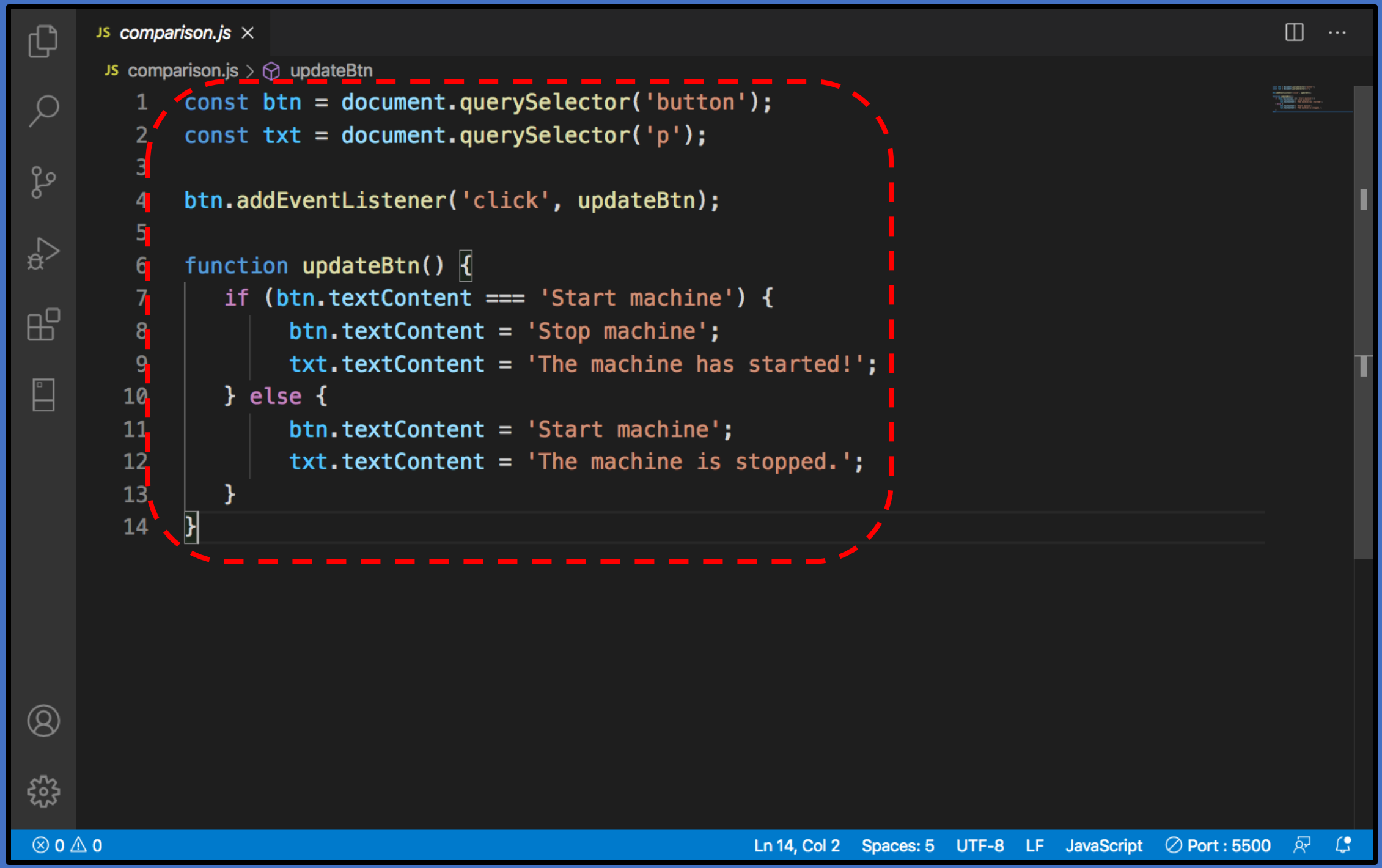
- **Sometimes** we will want to **run true/false tests**, then **act** accordingly **depending** on the **result** of that **test**.
  - ✓To do this we use **comparison operators**.

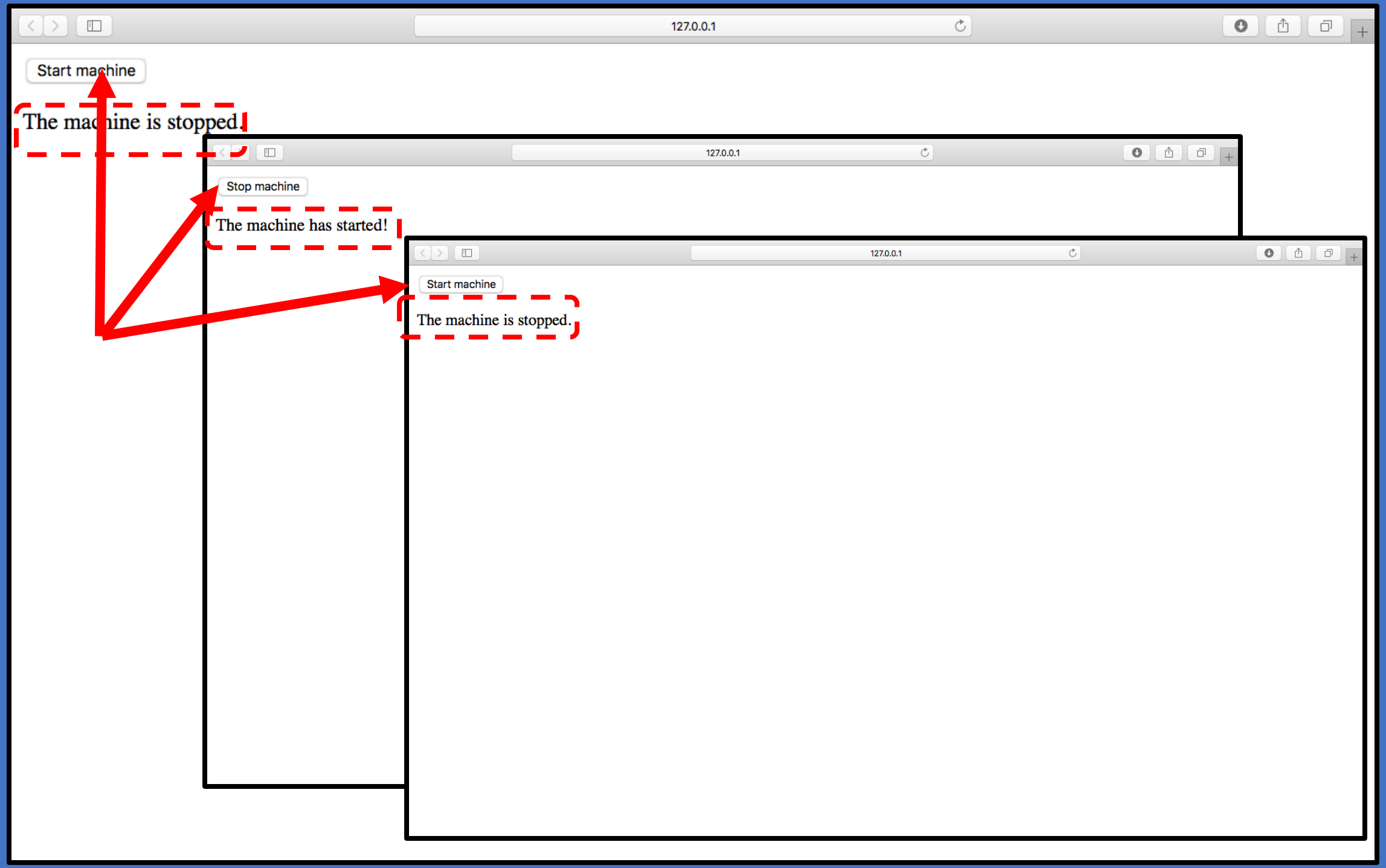
Operator	Name	Purpose	Example
`===`	Strict equality	Tests whether the left and right values are identical to one another	`5 === 2 + 4`
`!==`	Strict-non-equality	Tests whether the left and right values are **not** identical to one another	`5 !== 2 + 3`
`<`	Less than	Tests whether the left value is smaller than the right one.	`10 < 6`
`>`	Greater than	Tests whether the left value is greater than the right one.	`10 > 20`
`<=`	Less than or equal to	Tests whether the left value is smaller than or equal to the right one.	`3 <= 2`
`>=`	Greater than or equal to	Tests whether the left value is greater than or equal to the right one.	`5 >= 4`

Note

You may see some people using == and != in their **tests** for **equality** and **non-equality**. These are **valid operators** in **JavaScript**, but they **differ** from ===/!==. The **former versions test whether** the **values** are the **same but not whether** the **values' datatypes** are the **same**. The **latter**, **strict versions test** the **equality** of **both** the **values** and their **datatypes**. The **strict versions** tend to **result** in **fewer errors**, so we **recommend** you use them.


comparison.html
comparison.html > html
1 <!DOCTYPE html>
2 <html lang="en">
3
4 <head>
5 <meta charset="UTF-8">
6 <meta name="viewport" content="width=device-width, initial-scale=1.0">
7 <title>Document</title>
8 <script src="comparison.js" defer></script>
9 </head>
10
11 <body>
12 <button>Start machine</button>
13 The machine is stopped.
14 </body>
15
16 </html>
⊗ 0 ⚠ 0
Ln 16, Col 8
Spaces: 4
UTF-8
LF
HTML
Port : 5500


comparison.js
comparison.js > updateBtn
1 const btn = document.querySelector('button');
2 const txt = document.querySelector('p');
3
4 btn.addEventListener('click', updateBtn);
5
6 function updateBtn() {
7 if (btn.textContent === 'Start machine') {
8 btn.textContent = 'Stop machine';
9 txt.textContent = 'The machine has started!';
10 } else {
11 btn.textContent = 'Start machine';
12 txt.textContent = 'The machine is stopped.';
13 }
14 }
0 0
Ln 14, Col 2
Spaces: 5
UTF-8
LF
JavaScript
Port : 5500


127.0.0.1
Start machine
The machine is stopped.
127.0.0.1
Stop machine
The machine has started!
127.0.0.1
Start machine
The machine is stopped.

Test your skills

developer.mozilla.org

Test your skills: Math

Jump to section

This aim of this skill test is to assess whether you've understood our Basic math in JavaScript — numbers and operators article.

> **Note**: You can try out solutions in the interactive editors below, however it may be helpful to download the code and use an online tool such as CodePen, jsFiddle, or Glitch to work on the tasks.
>
> If you get stuck, then ask us for help — see the Assessment or further help section at the bottom of this page.

> **Note**: In the examples below, if there is an error in your code it will be outputted into the results panel on the page, to help you try to figure out the answer (or into the browser's JavaScript console, in the case of the downloadable version).

Math 1

https://mzl.la/36hTtbN

Questions?